
Advances in Approximate Inference for Logical Credal Networks

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Logical Credal Networks (LCNs) are an expressive probabilistic logic that com-
2 bines propositional formulas with imprecise probability bounds to represent sets
3 of probability distributions. Although LCNs allow for cyclic dependencies and
4 impose few syntactic restrictions, inference in these models remains challenging.
5 Existing exact methods scale exponentially with the number of propositions, while
6 the only available approximate approach relies on a single message-passing al-
7 gorithm, ARIEL. In this paper, we exploit the underlying chain graph structure
8 of LCNs and recast inference as a standard credal network problem. Based on
9 this formulation, we introduce three novel approximate inference algorithms that
10 apply either variable elimination or message-passing schemes propagating interval-
11 valued vector messages to compute outer probability bounds. In addition, we
12 develop a coordinate descent method that yields complementary inner bounds.
13 We evaluate the proposed algorithms on a collection of randomly generated and
14 realistic benchmark LCN instances. The results demonstrate substantial speedups
15 over exact inference while maintaining good quality probability bounds.

16 1 Introduction

17 Combining logical and probabilistic reasoning is a long-standing goal of artificial intelligence.
18 Frameworks such as Markov Logic Networks [?], ProbLog [?], and Probabilistic Soft Logic [?]
19 encode domain knowledge as logical rules augmented with uncertainty, but most of them require
20 *precise* probability values or impose strong structural assumptions such as acyclicity. In many real
21 applications, however, domain expertise yields only *ranges* of plausible probabilities and the rules of
22 interest are neither directed nor acyclic.

23 *Logical Credal Networks* (LCNs) [?] were introduced to address this gap: they allow *imprecise*
24 specifications of the form $\alpha \leq P(\phi \mid \varphi) \leq \beta$ over arbitrary propositional formulas, with no acyclicity
25 restriction. An LCN defines a *credal set* — the set of all distributions consistent with its constraints —
26 and inference amounts to computing tight lower and upper bounds on posterior probabilities of query
27 formulas. This flexibility makes LCNs well suited to causal reasoning with partial knowledge, robust
28 decision making, and trustworthy neuro-symbolic systems, but also makes inference considerably
29 harder than in Bayesian or classical credal networks. Existing LCN inference algorithms are limited
30 to exact constrained optimization [?], which is confined to small instances, and the ARIEL message-
31 passing scheme [?], which is currently the *only* approximate algorithm for LCN marginal inference
32 and itself struggles to scale. The lack of a richer toolbox of scalable approximate algorithms is the
33 main obstacle to deploying LCNs at practical sizes.

34 In this paper we revisit the chain graph factorization of LCNs established by [?] and turn it into a
35 concrete algorithmic program. The key observation is that the local Markov condition associated with
36 the LCN’s chain graph decomposes global LCN inference into a collection of *local credal network*

inference tasks, each of which can be tackled with the well-developed machinery of credal networks [? ? ? ?]. This bridge lets us import and extend credal-network algorithms into the LCN setting for the first time, trading a controlled relaxation — the strong extension $\mathbb{K}(\mathbf{X})$ is a superset of the LCN’s distribution set $\mathcal{P}(\mathbf{X})$ — for substantial scalability gains. Our contributions are:

1. A **chain-graph-to-credal-network bridge** that decomposes global LCN inference into local credal-network tasks and makes the relationship between $\mathcal{P}(\mathbf{X})$ and $\mathbb{K}(\mathbf{X})$ explicit as a principled relaxation.
2. Three new approximate inference algorithms covering **complementary bound regimes**: **CCTE**, a credal bucket-tree elimination scheme producing exact-on-the-credal-network bounds, together with a message-truncated variant CCTE_{cm} ; **IBP**, a fast interval-vector message-passing algorithm producing outer bounds with native multi-valued support; and **CDVE**, a coordinate descent algorithm producing inner bounds by optimizing over extreme points. All three come with explicit complexity analyses.
3. An **extensive empirical evaluation** on random LCNs, realistic LCNs from Bayesian network benchmarks, and a new family of chain-graph LCNs derived from the Supreme Court analysis of [?], which scales to $n=40$ propositions. CDVE yields the tightest bounds among credal-network methods — competitive with ARIEL and the only algorithm completing the largest instances — while IBP is consistently fastest and CCTE_{cm} offers a practical middle ground.

Beyond these immediate results, the bridge opens LCN inference to the broader credal-network literature and substantially widens the range of LCN instances reachable in practice.

2 Background

2.1 Logical Credal Networks

A Logical Credal Network (LCN) [?] consists of a set of finite set of propositions $\mathcal{A} = \{A_1, \dots, A_n\}$ and a set of constraints or probabilistic sentences $\mathcal{S}_{\mathcal{A}}$. Each proposition A_i is associated with a binary indicator variable X_i : if A_i holds in an interpretation then $X_i = 1$; otherwise, $X_i = 0$. Each probabilistic sentence in $\mathcal{S}_{\mathcal{A}}$ is a constraint of the form:

$$\alpha \leq P(\phi|\varphi) \leq \beta, \quad (1)$$

where ϕ and φ are propositional logic formulas over $\mathcal{A}$. The formula φ can be a tautology $\top$, yielding an unconditional constraint $\alpha \leq P(\phi) \leq \beta$.

Given an LCN, its *primal graph* is defined as follows. Each proposition is a node (a *proposition-node*). Each constraint $\alpha \leq P(\phi|\varphi) \leq \beta$ is processed as follows: (i) A node labeled with formula ϕ is added (a *formula-node*), and a directed edge is added from ϕ to each proposition in ϕ ; (ii) If $\varphi = \top$, an edge is added from each proposition in ϕ to node ϕ (creating bidirected edges); (iii) If $\varphi \neq \top$, a node labeled φ is added, a directed edge from φ to ϕ is added, and edges from each proposition in φ to φ are added [?].

The *structure* of an LCN is a graph that can be extracted from the primal graph as follows: (i) For each formula-node ϕ appearing as a conditioned formula in a constraint place an undirected edge between any two propositions in ϕ ; (ii) For each pair of formula-nodes φ and ϕ in a constraint, add a directed edge from each proposition in φ to each proposition in ϕ ; (iii) If there is now a pair of edges $A \rightrightarrows B$ for some propositions A, B , replace both by an undirected edge; (iv) Remove formula-nodes and all their edges; remove duplicate edges [?].

The parents of a proposition node A_i in G , denoted $\text{pa}(A_i)$, are the proposition nodes that reach A_i via a directed path whose intermediate nodes (if any) are formula nodes. The descendants of A_i , denoted $\text{de}(A_i)$, are the proposition nodes reachable from A_i via a directed path whose intermediate nodes do not belong to $\text{pa}(A_i)$. An LCN is endowed with a Local Markov Condition (LMC).

Definition 1 (LMC(LCN)). *Given an LCN $\mathcal{L}$ with primal graph G , every proposition A_i in $\mathcal{L}$ is independent of its non-parent non-descendant propositions conditioned on A_i ’s parents in G .*

Example 1. Figure 1 shows a simple LCN with 5 propositions and 5 constraints. The corresponding primal graph and structure are given in Figures 1b and 1c, respectively. The LMC implies the

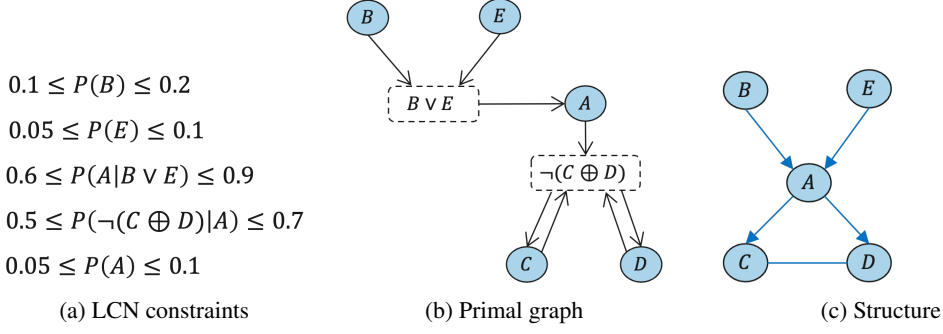


Figure 1: A simple LCN instance with its primal graph and corresponding structure.

86 following conditional independencies: B and E are marginally independent, C is conditionally
87 independent of B and E given C 's parents $\{A, D\}$, and D is conditionally independent of B and E
88 given $\{A, C\}$, respectively.

89 **Inference in LCNs.** Exact inference in an LCN with n propositions amounts to computing lower
90 and upper posterior bounds for a query formula ψ via a non-linear optimization problem over the 2^n
91 interpretation probabilities, subject to normalization, linear constraints encoding the probabilistic
92 sentences, and non-linear constraints corresponding to the conditional independencies induced by
93 the local Markov condition, while optimizing a linear objective representing $\mathbb{P}(\psi)$ [?]. Although
94 principled, this approach is limited to small instances. To address scalability, ARIEL [?] proposes
95 an approximate message-passing algorithm on a factor-graph representation of the LCN, where
96 variable-to-factor messages propagate probability bounds and factor-to-variable messages refine
97 them by solving local constrained non-linear programs under additional independence assumptions
98 encoded as quadratic equalities. Empirically, ARIEL scales to larger problems with high-quality
99 bounds, but its performance remains constrained by the complexity of these local optimizations and
100 it is currently the only available approximation method for marginal inference in LCNs.

101 2.2 Chain Graphs and Factorization

102 **Definition 2** (chain graph). Let $G = (V, E)$ be a mixed graph with $E = E_{\rightarrow} \cup E_{-}$, where $E_{\rightarrow}$ is
103 a set of directed edges and E_{-} a set of undirected edges. The graph G is called a chain graph if
104 there exists a partition $V = C_1 \dot{\cup} C_2 \dot{\cup} \dots \dot{\cup} C_k$ such that: (i) for each i , the subgraph induced by
105 C_i contains only undirected edges: $\forall u, v \in C_i, (u, v) \in E \implies \{u, v\} \in E_{-}$; (ii) for $i \neq j$, every
106 edge between C_i and C_j is directed, and all such edges have the same orientation; (iii) the directed
107 graph obtained by contracting each C_i to a single vertex is acyclic. The sets $C_1, \dots, C_k$ are called
108 the chain components of G .

109 Let G be a chain graph with nodes $\mathcal{N}$. If there is a directed edge from A to B in G then A is a *parent*
110 of B and B is a *child* of A . The parents of A are denoted $\text{pa}(A)$. If there is an undirected edge
111 $A - B$ then both nodes are said to be *neighbors*. The neighbors of A are denoted by $\text{ne}(A)$. The
112 *boundary* of a node A , denoted $\text{bd}(A)$, is the set $\text{pa}(A) \cup \text{ne}(A)$. The boundary of a set $\mathcal{B}$ of nodes
113 is $\text{bd}(\mathcal{B}) = \bigcup_{A \in \mathcal{B}} \text{bd}(A) \setminus \mathcal{B}$. The *Local Markov Condition* (LMC) for chain graphs states:

114 **Definition 3** (LMC(C)). A node A is independent, given its boundary $\text{bd}(A)$, of all nodes that are
115 not A itself nor descendants of A nor boundary nodes of A .

116 Assuming positive probabilities, the local and global Markov conditions are equivalent and yield a
117 *factorization property* for chain graphs [? ?]. Let $T_1, \dots, T_n$ be the chain components ordered so
118 that nodes in T_i can only be at the end of directed edges from earlier components. Then:

$$\mathbb{P}(\mathbf{X} = \mathbf{x}) = \prod_{i=1}^n \mathbb{P}(\mathcal{N}_i = \mathbf{x}_{\mathcal{N}_i} \mid \text{bd}(\mathcal{N}_i) = \mathbf{x}_{\text{bd}(\mathcal{N}_i)}) \quad (2)$$

119 where $\mathcal{N}_i$ is the set of nodes in the i th chain component, and $\mathbf{x}_{\mathcal{N}_i}, \mathbf{x}_{\text{bd}(\mathcal{N}_i)}$ are the projections of $\mathbf{x}$
120 over $\mathcal{N}_i$ and $\text{bd}(\mathcal{N}_i)$. It follows that if the structure of a given LCN $\mathcal{L}$ is a chain graph, then the Local

121 Markov Conditions from Definitions 1 and 3 are equivalent [?]. Furthermore, any joint probability
 122 distribution represented by $\mathcal{L}$ admits a chain graph factorization [?].

123 3 From LCNs to Credal Networks

124 In this section, we show how the chain-graph factorization of an LCN can be leveraged to reduce
 125 LCN inference to inference in a credal network [?]. We therefore focus on LCNs whose underlying
 126 structure is a chain graph, which we refer to as *chain-graph LCNs*. We begin with a brief review of
 127 credal networks.

128 3.1 Credal Networks

129 A *credal network* (CN) [?] is defined by a tuple $\langle \mathbf{X}, \mathbf{D}, \mathbf{K}, G \rangle$, where $\mathbf{X} = \{X_1, \dots, X_n\}$ is a set
 130 of discrete variables with finite domains $\mathbf{D} = \{D_1, \dots, D_n\}$, G is a directed acyclic graph (DAG)
 131 over $\mathbf{X}$ as nodes, and $\mathbf{K} = \{\mathbb{K}(X_i | \mathbf{pa}(X_i) = \pi_{ik})\}$ is a set of separately specified conditional credal
 132 sets for each variable X_i and each k -th configuration π_{ik} of its parents Π_i in G . A credal set is a set
 133 of probability distributions [?] and we consider credal sets that are closed and convex with a finite
 134 number of vertices (e.g., probability intervals). The *strong extension* $\mathbb{K}(\mathbf{X})$ of a credal network is the
 135 *convex hull* (denoted CH) of all joint distributions that satisfy the following Markov property: every
 136 variable is strongly independent of its non-descendants conditional on its parents [?].

$$\mathbb{K}(\mathbf{X}) = CH\{\mathbb{P}(\mathbf{X}) : \mathbb{P}(\mathbf{X}) = \prod_{i=1}^n \mathbb{P}(X_i | \pi_{ik}), \mathbb{P}(X_i | \pi_{ik}) \text{ is a vertex of } \mathbb{K}(X_i | \mathbf{pa}(X_i) = \pi_{ik})\} \quad (3)$$

137 It can be shown that the strong extension $\mathbb{K}(\mathbf{X})$ can be built from the extreme points of the conditional
 138 local credal sets denoted by $\text{ext}(\mathbb{K}(X_i | \mathbf{pa}(X_i) = \pi_{ik}))$ [?].

139 3.2 Chain Graph Approximation for LCNs

140 Let $\mathcal{L}$ be a chain-graph LCN with n variables $\mathbf{X} = \{X_1, \dots, X_n\}$, representing a set of m joint
 141 distributions $\mathcal{P}(\mathbf{X}) = \{\mathbb{P}_1(\mathbf{X}), \mathbb{P}_2(\mathbf{X}), \dots, \mathbb{P}_m(\mathbf{X})\}$, where each distribution $\mathbb{P}_i(\mathbf{X})$ satisfies the
 142 probabilistic sentences of $\mathcal{L}$ as well as its local Markov condition. Moreover, since the structure
 143 of $\mathcal{L}$ is a chain graph with components $\mathcal{T} = \{T_1, \dots, T_k\}$ such that $T_j \subseteq \mathbf{X}$ for $1 \leq j \leq k$, each
 144 distributions $\mathbb{P}_i(\mathbf{X})$ admits the following factorization [?]:

$$\mathbb{P}_i(\mathbf{X} = \mathbf{x}) = \prod_{T_j \in \mathcal{T}} \mathbb{P}_i(T_j = \mathbf{x}_{T_j} | \mathbf{pa}(T_j) = \mathbf{x}_{\mathbf{pa}(T_j)}) \quad (4)$$

145 where $\mathbf{x}_{T_j}$ and $\mathbf{x}_{\mathbf{pa}(T_j)}$ are the projections of $\mathbf{x}$ over T_j and $\mathbf{pa}(T_j)$, respectively.

146 One way to approximate the set of joint distributions $\mathcal{P}(\mathbf{X})$ represented by $\mathcal{L}$ is via the strong
 147 extension $\mathbb{K}(\mathbf{X})$ of a credal network $\mathcal{C}$ induced by the chain components of the structure of $\mathcal{L}$. In this
 148 approach, $\mathcal{C}$ is defined as a DAG G whose nodes correspond to the chain components $\{T_1, \dots, T_k\}$.
 149 Each chain component T_j is represented by a variable Y_j with domain $D_j = \prod_{X_i \in T_j} D_i$, that is, the
 150 Cartesian product of the domains of the variables in T_j . If T_j contains more than one variable, the
 151 corresponding Y_j is called a *compound variable*.

152 For each variable Y_j and each configuration π_{jk} of its parents $\mathbf{pa}(Y_j)$ in G , the approximation is
 153 completed by specifying a local conditional credal set $\mathbb{K}(Y_j | \mathbf{pa}(Y_j) = \pi_{jk})$. Specifically, for each
 154 configuration $\mathbf{y}$ of the variables $\mathbf{Y} = \{Y_j\} \cup \mathbf{pa}(Y_j)$ we obtain lower and upper bounds $[lb, ub]$
 155 on the conditional probability $\mathbb{P}(Y_j = \mathbf{y}_{Y_j} | \mathbf{pa}(Y_j) = \mathbf{y}_{\mathbf{pa}(Y_j)})$ by solving a local fractional linear
 156 program defined by the probabilistic sentences of $\mathcal{L}$ involving only the variables $Y_j \cup \mathbf{pa}(Y_j)$.

157 Let $\vec{p} = (p_1, \dots, p_M)$ be a vector representing the probabilities of the $M = 2^m$ configurations
 158 of variables $\mathbf{Y}$, and let $\vec{A}_\varphi = (a_1^\varphi, \dots, a_M^\varphi)$ be a binary *indicator vector* vector, where $a_j^\varphi = 1$
 159 iff formula φ is true in the j -th configuration and 0 otherwise. Since the $P(\varphi)$ is the sum of
 160 the probabilities of the configurations satisfying φ , we have $P(\varphi) = \vec{A}_\varphi \odot \vec{p}$ where $\odot$ is the
 161 dot-product of two vectors. Therefore, we solve the fractional LP defined by Equations 5–9,

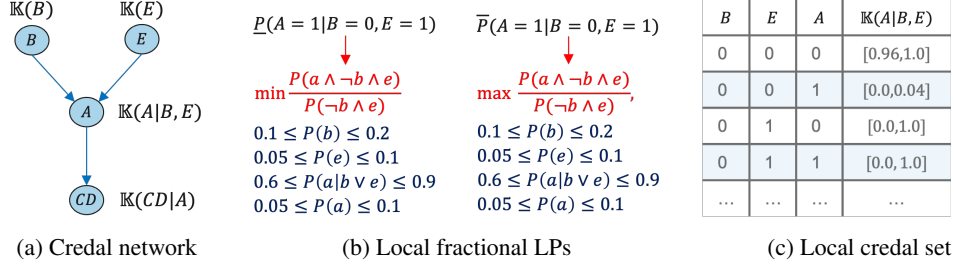


Figure 2: The credal network for the chain-graph factorization of the LCN from Figure 1.

where $S_j = \{s_{j1}, \dots, s_{jk}\}$ and $T_j = \{t_{j1}, \dots, t_{jl}\}$ are the literals representing the positive or negative assignments of the singleton variables Y_j and $\text{pa}(Y_j)$ in $\mathbf{y}$, $\alpha \leq P(\phi) \leq \beta$ and $\alpha \leq P(\phi | \varphi) \leq \beta$ are the probabilistic sentences defined over variables $\mathbf{Y}$, $\vec{A}_\phi$ and $\vec{A}_{\phi \wedge \varphi}$ are the indicator vectors for formulas ϕ and $\phi \wedge \varphi$, and $\vec{A}_\psi, \vec{A}_\rho$ are the indicator vectors corresponding to the formulas $\psi = (s_{j1} \wedge \dots \wedge s_{jk} \wedge t_{j1} \wedge \dots \wedge t_{jl})$ and $\rho = (t_{j1} \wedge \dots \wedge t_{jl})$, respectively.

Example 2. The structure of the LCN from Figure 1c is a chain graph. In Figure 2a we show the credal network induced by the chain-graph factorization. In this case, any distribution $\mathbb{P}(\cdot)$ factorizes as: $\mathbb{P}(A, B, C, D, E) = \mathbb{P}(B) \cdot \mathbb{P}(E) \cdot \mathbb{P}(A|B, E) \cdot \mathbb{P}(C, D|A)$. Figure 2b shows the fractional linear programs for computing lower and upper bounds on $P(A = 1|B = 0, E = 1)$, respectively. The credal set $\mathbb{K}(A|B, E)$ is given in Figure 2c.

In summary, our chain graph approximation for LCNs consists of the following steps: (i) extract the structure of the LCN and identify the chain components $\{T_1, \dots, T_k\}$; (ii) construct a DAG G with nodes corresponding to each T_j ; (iii) for each T_j and its parents $\text{pa}(T_j)$ in G identify the probabilistic sentences of the LCN that are defined on variables $Y_j \cup \text{pa}(Y_j)$ only; (iii) compute the corresponding conditional credal sets $\mathbb{K}(Y_j | \text{pa}(Y_j) = \pi_{jk})$ by solving the local fractional linear programs given by Equations 5–9 (in practice we solve linear programs by applying the Charnes-Cooper transform [?]). The result is a credal network whose strong extension approximates the set of distributions represented by the input LCN. Consequently, standard credal network inference algorithms can be applied to compute bounds on the posterior marginals of the original LCN.

RM: Add one sentence about the relationship between $\mathcal{P}(\mathbf{X})$ and $\mathbb{K}(X)$

3.3 Credal Cluster Tree Elimination

Next we present the first of three inference algorithms that leverage the chain graph factorization of an LCN to compute marginal probability bounds. Credal Cluster Tree Elimination (CCTE) extends Credal Variable Elimination (CVE) [?] to cluster (bucket) trees [? ? ?] by propagating credal potentials, represented as finite sets of non-negative functions [?].

Definition 4 (potential). A potential $\phi(\mathbf{Y})$ over variables $\mathbf{Y}$ is a finite set of non-negative real-valued functions $\{f_1, f_2, \dots, f_m\}$ on the joint domain of $\mathbf{Y}$. The product (or combination) of two potentials is defined by $\phi \cdot \psi = \{f \cdot g : f \in \phi, g \in \psi\}$, where $(f \cdot g)(\mathbf{y}, \mathbf{z}) = f(\mathbf{y}) \cdot g(\mathbf{z})$ over the joint domain $\mathbf{Y} \cup \mathbf{Z}$. The sum-marginal is defined by $\sum_X \phi(\mathbf{Y}) = \{\sum_x f(\mathbf{y}) : f \in \phi\}$, producing functions over $\mathbf{Y} \setminus \{X\}$. A pruning operation is defined by $\text{prune}(\phi) = \{f \in \phi : \nexists g \in \phi \setminus \{f\} \text{ s.t. } g(\mathbf{y}) \geq f(\mathbf{y}) \forall \mathbf{y}\}$.

Algorithm 4 outlines CCTE. Given an elimination order $\mathbf{o} = (Y_1, \dots, Y_m)$ and initial potentials (obtained from the extreme points of each conditional credal set $\mathbb{K}(Y_j | \text{pa}(Y_j))$ and evidence), each potential is assigned to the bucket of the first variable in its scope. For bucket Y_i , the message scope is the union of scopes of all assigned potentials and incoming messages, excluding Y_i . The

Algorithm 1 Credal Cluster Tree Elimination (CCTE)

Require: LCN $\mathcal{L}$ with variables $\{X_1, \dots, X_n\}$, evidence $\mathbf{E} = \mathbf{e}$

Ensure: Bounds $[P(X_i=x|\mathbf{e}), \bar{P}(X_i=x|\mathbf{e})]$ for non-evidence variables X_i and states x

- 1: Build credal network $\mathcal{C}$ using chain graph factorization of $\mathcal{L}$
- 2: Build potentials from extreme points of each $\mathbb{K}(Y_j \mid \mathbf{pa}(Y_j))$ and evidence indicators
- 3: Build bucket tree $\mathcal{T}$: assign potentials, determine parent-child
- 4: **for** each Y_i in $\mathcal{T}$ (leaves to root) **do**
- 5: $\psi_i \leftarrow \prod \{\text{pots}(Y_i) \cup \{\lambda_{Y_j} : Y_j \in \text{ch}(Y_i)\}\}$
- 6: $\lambda_{Y_i} \leftarrow \text{prune}(\sum_{Y_i} \psi_i)$
- 7: Send λ_{Y_i} to parent
- 8: **for** each Y_i in $\mathcal{T}$ (root to leaves) **do**
- 9: **for** each child Y_j of Y_i in $\mathcal{T}$ **do**
- 10: $\psi \leftarrow \prod \{\pi_{Y_i}, \text{pots}(Y_i), \{\lambda_{Y_k} : Y_k \in \text{ch}(Y_i) \setminus \{Y_j\}\}\}$
- 11: $\pi_{Y_j} \leftarrow \text{prune}(\sum_{Y_j} \psi)$
- 12: Send π_{Y_j} to child Y_j
- 13: **for** each variable Y_i **do**
- 14: $\Phi_i \leftarrow \prod \{\text{pots}(Y_i), \{\lambda_{Y_j} : Y_j \in \text{ch}(Y_i)\}, \pi_{Y_i}\}$
- 15: $\phi_i \leftarrow \sum_{\mathbf{Y} \setminus \{Y_i\}} \Phi_i$
- 16: $\underline{P}(Y_i=y) \leftarrow \min_{f \in \phi_i} f(y) / \sum_{y'} f(y')$
- 17: $\bar{P}(Y_i=y) \leftarrow \max_{f \in \phi_i} f(y) / \sum_{y'} f(y')$

parent of Y_i is the next variable in $\mathbf{o}$ appearing in this scope, inducing a rooted bucket tree with root Y_m . The algorithm processes the buckets in an upward pass, from leaves to the root following the elimination order. At bucket Y_i , combine local potentials with incoming messages from children to form ψ_i , marginalize Y_i , prune, and send $\lambda_{Y_i} = \text{prune}(\sum_{Y_i} \psi_i)$ to the parent. Subsequently, in a downward pass, from root to leaves, for each child bucket Y_j of Y_i , combine the downward message from the parent, local potentials, and all upward messages except from Y_j , marginalize Y_i , prune, and send π_{Y_j} to Y_j . After the message distribution, the information at bucket Y_i is $\Phi_i = (\text{local potentials}) \cdot \prod_{j \in \text{children}(i)} \lambda_{Y_j} \cdot \pi_{Y_i}$. Marginalizing all variables except Y_i , normalizing, and taking component-wise minima and maxima yields lower and upper bounds for Y_i .

While CCTE preserves tight bounds, intermediate potentials can grow exponentially. Following [?], we can approximate the potentials by fixing their cardinality to a relatively small number B . Specifically, we group the elements of $\phi(\mathbf{Y})$ into B clusters based on minimizing the Manhattan distance to each cluster's centroid (i.e., minimize $\sum_{i=1}^m |p_i - r_i|$, where p_i and r_i are the i -th components of p and r , respectively). Then, for each cluster we replace its components with their mean values. This yields an approximation methods denoted hereafter by CCTE_{cm}.

Theorem 1 (complexity). *The time and space complexity of CCTE is $O(m \cdot C \cdot d^{w^*})$ where m is the number of variables, d bounds the domain size, w^* is the treewidth, and C is the maximum potential cardinality (which can be exponential). With potential approximation, C is bounded by B [?].*

Since some Y_j are compound variables, posterior lower and upper bounds for the corresponding singleton variables X_{jk} can be obtained efficiently by solving a small linear program over the compound marginal polytope [?].

3.4 Interval Belief Propagation

Our second approach, called Interval Belief Propagation (IBP) and outlined in Algorithm 2, is an iterative message-passing algorithm for approximate marginal inference that propagates *interval messages* on the *factor graph* associated with the credal network $\mathcal{C}$ corresponding to the chain-graph factorization of the input LCN. Unlike previous algorithms like L2U [?] and GL2U [?] which are restricted to binary variables, IBP natively handles multi-valued variables.

The *factor graph* of a credal network $\mathcal{C}$ is a bipartite graph with variable nodes and factor nodes. Each variable Y_j in $\mathcal{C}$ corresponds to a variable node with cardinality $|D_j|$. Each conditional credal set $\mathbb{K}(Y_i \mid \mathbf{pa}(Y_j))$ defines a factor node f_j whose scope is $\{Y_j\} \cup \mathbf{pa}(Y_j)$. The factor stores the extreme points of $\mathbb{K}(Y_j \mid \pi_{jk})$ for all parent configurations π_{jk} . For a variable X with k states, an *interval message* is a pair of vectors $(\mathbf{l}, \mathbf{u}) \in \mathbb{R}^k \times \mathbb{R}^k$ where $0 \leq l_x \leq u_x \leq 1$ for each state x . Messages are of two types: variable-to-factor $\mu_{v \rightarrow f}$ and factor-to-variable $\mu_{f \rightarrow v}$

Algorithm 2 Interval Belief Propagation (IBP)

Require: LCN $\mathcal{L}$ with variables $\{X_1, \dots, X_n\}$, evidence $\mathbf{E} = \mathbf{e}$, iterations T , threshold τ

Ensure: Bounds $[\underline{P}(Y_i=y|\mathbf{e}), \bar{P}(Y_i=y|\mathbf{e})]$ for non-evidence variables Y_i and states y

- 1: Build credal network $\mathcal{C}$ using chain graph factorization of $\mathcal{L}$
- 2: Build factor graph and initialize factors from extreme points of each $\mathbb{K}(Y_j \mid \mathbf{pa}(Y_j))$
- 3: Set $\mu_{v \rightarrow f} \leftarrow (\mathbf{0}, \mathbf{1})$ and $\mu_{f \rightarrow v} \leftarrow (\mathbf{0}, \mathbf{1})$
- 4: **for** each evidence variable $E_i = e_i$ **do**
- 5: Set $\mu_{E_i \rightarrow f} \leftarrow (\delta_{e_i}, \delta_{e_i}) \forall f \in \mathcal{N}(Y_j)$
- 6: **for** iteration $t = 1, \dots, T$ **do**
- 7: $\delta \leftarrow 0$
- 8: $\triangleright$ VARIABLE-TO-FACTOR UPDATES
- 9: **for** each $v (\notin \mathbf{E})$, each $f \in \mathcal{N}(v)$ **do**
- 10: **for** each value x **do**
- 11: $l_{v \rightarrow f}(x) \leftarrow \max_{f' \in \mathcal{N}(v) \setminus \{f\}} l_{f' \rightarrow v}(x)$
- 12: $u_{v \rightarrow f}(x) \leftarrow \min_{f' \in \mathcal{N}(v) \setminus \{f\}} u_{f' \rightarrow v}(x)$
- 13: Update δ
- 14: $\triangleright$ FACTOR-TO-VARIABLE UPDATES
- 15: **for** each f_i , each variable $v \in \text{scope}(f_i)$ **do**
- 16: **for** each vertex combination $\mathbf{vc}$ **do**
- 17: **for** each corner-point combination $\mathbf{cc}$ **do**
- 18: Compute $\mathbf{m}(v)$ using $\mathbf{vc}$ and $\mathbf{cc}$
- 19: $\mathbf{p} \leftarrow \mathbf{m} / \sum_x m(x)$
- 20: $\mathbf{l}_{\text{new}} \leftarrow \min(\mathbf{l}_{\text{new}}, \mathbf{p})$
- 21: $\mathbf{u}_{\text{new}} \leftarrow \max(\mathbf{u}_{\text{new}}, \mathbf{p})$
- 22: $\mu_{f \rightarrow v} \leftarrow (\mathbf{l}_{\text{new}}, \mathbf{u}_{\text{new}})$
- 23: Update δ
- 24: **if** $\delta < \tau$ **then break**
- 25: $\triangleright$ EXTRACT MARGINAL BOUNDS
- 26: **for** each non evidence variable Y_i **do**
- 27: **for** each value $y \in D_{Y_i}$ **do**
- 28: $\underline{P}(Y_i=y \mid \mathbf{E}=\mathbf{e}) \leftarrow \max_{f \in \mathcal{N}(Y_i)} l_{f \rightarrow Y_i}(y)$
- 29: $\bar{P}(Y_i=x \mid \mathbf{E}=\mathbf{e}) \leftarrow \min_{f \in \mathcal{N}(Y_i)} u_{f \rightarrow Y_i}(y)$
- 30: **return** $[\underline{P}(Y_i=y \mid \mathbf{E}=\mathbf{e}), \bar{P}(Y_i=y \mid \mathbf{E}=\mathbf{e})]$

which are initialized to the vacuous interval $([0, \dots, 0], [1, \dots, 1])$. Evidence variables have their messages clamped to the one-hot indicator $\delta_{e_j} \in \{0, 1\}^{|D_{E_j}|}$ (entry e_j equals 1, all others equal 0):

$\mu_{E_j \rightarrow f} \leftarrow (\delta_{e_j}, \delta_{e_j}) \forall f \in \mathcal{N}(E_j)$.

Variable-to-Factor Updates. Variable v sends to factor f the *intersection* of messages from all other neighboring factors:

$$l_{v \rightarrow f}(x) = \max_{f' \in \mathcal{N}(v) \setminus \{f\}} l_{f' \rightarrow v}(x), \quad u_{v \rightarrow f}(x) = \min_{f' \in \mathcal{N}(v) \setminus \{f\}} u_{f' \rightarrow v}(x) \quad (10)$$

with the constraint $u_{v \rightarrow f}(x) \geq l_{v \rightarrow f}(x)$ enforced.

Factor-to-Variable Updates. The message from factor f_i to target variable v is computed by scanning over all *vertex combinations* and *corner-point combinations*.

Definition 5 (vertex combination). A vertex combination $\mathbf{vc} = (j_1, \dots, j_m)$ selects one extreme point per parent configuration of factor f_i , specifying a complete conditional distribution $P_{\mathbf{vc}}(Y_i \mid \pi_k)$ for every parent configuration π_k . The total number is $\prod_{k=1}^m n_{\pi_k}$.

Definition 6 (corner-point combination). A corner-point combination $\mathbf{cc} = (c_1, \dots, c_{|\mathcal{O}|})$ with $c_j \in \{0, 1\}$ selects either the lower or upper bound vector for each “other” variable $w_j \in \mathcal{O} = \text{scope}(f_i) \setminus \{v\}$. The total number is $2^{|\mathcal{O}|}$.

For each $(\mathbf{vc}, \mathbf{cc})$ pair, the unnormalized marginal over v is computed as:

Case 1 ($v = Y_i$, target is child):

$$m(v=y) = \sum_{\pi} P_{\mathbf{vc}}(Y_i=y \mid \pi) \cdot \prod_{w \in \mathbf{pa}(Y_i) \cap \mathcal{O}} \mathbf{q}_w(\pi_w). \quad (11)$$

Case 2 ($v \in \mathbf{pa}(Y_i)$, target is parent):

$$m(v=y) = \sum_{\pi: \pi_v=y} \left(\sum_{y_i} P_{\mathbf{vc}}(Y_i=y_i \mid \pi) \cdot \mathbf{q}_{Y_i}(y_i) \right) \cdot \prod_{w \in \mathbf{pa} \setminus \{v\}} \mathbf{q}_w(\pi_w). \quad (12)$$

After normalization, the factor-to-variable message takes component-wise min/max over all pairs:

$l_{f \rightarrow v}(y) = \min_{\mathbf{vc}, \mathbf{cc}} p(y), \quad u_{f \rightarrow v}(y) = \max_{\mathbf{vc}, \mathbf{cc}} p(y)$.

Algorithm 3 Coordinate Descend with Variable Elimination (CDVE)

Require: LCN $\mathcal{L}$ with variables $\{X_1, \dots, X_n\}$, evidence $\mathbf{E} = \mathbf{e}$, iterations T
Ensure: Bounds $[\underline{P}(Z=z|\mathbf{e}), \bar{P}(Z=z|\mathbf{e})]$ for non-evidence variables Z and states z

```

7:  for sweep  $t = 1, \dots, T$  do
8:    for each  $(X_i, \pi_i)$  do
9:       $v^* \leftarrow \arg \text{sense}_{v \in \text{ext } K(X_i|\pi_i)} \text{VE}(\{P\}[P(X_i | \pi_i) \leftarrow v], Z, \mathbf{y})(z)$ 
         $\triangleright z = \text{target state this run is tracking}$ 
10:     if  $v^*$  strictly improves  $p^*(z)$  then
         $P(X_i | \pi_i) \leftarrow v^*, p^* \leftarrow \text{VE}(\{P\}, Z, \mathbf{y})$ 
11:     if sweep made no change then break
12:      $\underline{p} \leftarrow \min(\underline{p}, p^*), \bar{p} \leftarrow \max(\bar{p}, p^*)$ 
13:  return  $[\underline{p}(z), \bar{p}(z)]$  for every  $z \in D_Z$ 

```

Marginal Extraction. After convergence, the lower and upper bounds for every non evidence variable Y_i are: $\underline{P}(y_i|\mathbf{e}) = \max_{f \in \mathcal{N}(Y_i)} l_{f \rightarrow Y_i}(y_i)$, $\bar{P}(y_i|\mathbf{e}) = \min_{f \in \mathcal{N}(Y_i)} u_{f \rightarrow Y_i}(y_i)$, respectively.

IBP produces *outer bounds*. On polytree-structured networks, it converges to exact bounds. On loopy graphs, convergence to a fixed point is empirically observed but not guaranteed. The complexity per iteration is $O(N_f \cdot V_{\max} \cdot 2^{S_{\max}} \cdot k^{S_{\max}})$, where N_f is the number of factors, $V_{\max}$ the maximum vertex combinations per factor, $S_{\max}$ the maximum scope size, and k the maximum domain size.

3.5 Coordinate Descent with Variable Elimination

Our third approach is called Coordinate Descent with Variable Elimination (CDVE) and is described by Algorithm 3. CDVE extends the ApproxLP method [?] that reformulates credal marginal inference as a multilinear program and solves it via coordinate descent.

Specifically, for every non-evidence variable Z , the CDVE runs coordinate descent twice — once with sense min and once with sense max. Each run starts by initializing all $P(X_i | \pi_i)$ at the centroid of their respective credal sets and computing $p^* = \text{VE}(\{P\}, Z, \mathbf{y})$, the full posterior vector on D_Z under these fixed distributions. A *sweep* then visits every (X_i, π_i) : for each vertex $v \in \text{ext } K(X_i | \pi_i)$, it temporarily sets $P(X_i | \pi_i) \leftarrow v$, re-evaluates VE, and accepts v (strictly, else rejects) if the target component $p^*(z)$ moves in the sense’s direction. Sweeps repeat until one produces no improvement or the cap T is reached. Because each VE call returns a full vector, the two runs together yield componentwise bounds $[\underline{p}(z), \bar{p}(z)]$ for every state $z \in D_Z$ simultaneously. Evidence variables are skipped by a point-mass shortcut. The output is *inner bounds* — coordinate descent explores only a subset of feasible vertex combinations, so the returned interval is contained in the true interval.

Theorem 2 (complexity). *The time and space complexity of CDVE is $O(T \cdot V_{\text{total}} \cdot n \cdot k^{w^*})$ and $O(n \cdot k^{w^*})$, where V_{total} is the total number of extreme points across all credal sets, k bounds the domain size, and w^* is the induced width.*

4 Experiments

We empirically evaluate the proposed schemes for marginal inference in LCNs and compare them against the exact inference method of [?] and the approximate ARIEL method [?] across several classes of LCNs.

Benchmarks. We consider three families of instances. (i) *Random LCNs* are generated with n propositions $\{X_1, \dots, X_n\}$ whose primal graph is a polytree or a dag, and whose sentences are of three types: $l \leq P(\varphi) \leq u$, $l \leq P(x_i | \varphi) \leq u$, and $l \leq P(\neg x_i | \varphi) \leq u$, where φ is a random formula over up to three propositions and the bounds l, u are sampled uniformly at random with $u - l \leq 0.6$; we ensure that every instance with $n \leq 10$ is consistent. (ii) *Realistic LCNs* are the benchmarks introduced in [?], which are derived from real-world Bayesian networks [?] and contain up to 10 propositions and up to 24 sentences of the form $l \leq P(x_i) \leq u$ and $l \leq P(x_i | \pi_i) \leq u$,

where $\pi_i = y_{i1} \wedge \dots \wedge y_{ik}$ is a parent configuration of X_i in the underlying Bayesian network.
 (iii) *Supreme Court LCNs* are a new family of instances derived from the causal chain graphs used in the US Supreme Court analysis of [?], designed specifically to stress scalability (up to $n=40$ propositions).

Algorithms and metrics. We compare six algorithms: the four credal-network inference schemes developed in this paper — CCTE (exact on the credal network), CCTE_{cm} (CCTE with message cardinality capped at 10; outer bounds), IBP (outer bounds), and CDVE (inner bounds) — together with the LCN-native Exact (exact) and ARIEL (approximate) algorithms of [? ?], which serve as references. For each instance and algorithm we report the average CPU time in seconds and the mean absolute error (MAE) of the posterior marginal bounds, separately for the lower and upper bound:

$$\text{MAE}_l = \frac{1}{n} \sum_{i=1}^n |\underline{P}^*(x_i) - \underline{P}(x_i)|, \quad \text{MAE}_u = \frac{1}{n} \sum_{i=1}^n |\overline{P}^*(x_i) - \overline{P}(x_i)|,$$

where $\underline{P}(x_i), \overline{P}(x_i)$ are the bounds produced by the evaluated algorithm and $\underline{P}^*(x_i), \overline{P}^*(x_i)$ are the *reference* bounds, i.e. the bounds produced by whichever algorithm we treat as ground truth for that benchmark — Exact whenever it is tractable, and ARIEL otherwise. The reference algorithm used in each experiment is stated explicitly below.

Hardware and software. All algorithms were implemented in Python 3.12 and used the `ipopt` 3.14 solver [?] with default settings for the linear and non-linear programs. All experiments were run on a 3.2 GHz Intel Core machine with 64 GB of RAM.

Results on random LCNs: small DAGs. Figure 3 plots the errors (top) and running times (bottom) as a function of problem size on dag LCNs with up to 10 variables. In this regime the Exact algorithm is tractable, so we use it as the reference. ARIEL is the overall tightest — consistent with [?] — and among the credal-network methods CDVE is the tightest, followed by CCTE and CCTE_{cm}, with IBP the loosest. The ranking reflects the theoretical guarantees: CCTE is exact on the credal network, CDVE computes inner bounds, and IBP computes outer bounds. Importantly, the residual gap between ARIEL and the credal-network methods does *not* reflect algorithmic looseness but rather the intrinsic cost of the chain-graph-to-credal-network relaxation: the strong extension $\mathbb{K}(\mathbf{X})$ is a strict superset of the LCN’s distribution set $\mathcal{P}(\mathbf{X})$, so any algorithm operating on the former is bound to be at least as loose as one operating on the latter. On running times, IBP is several orders of magnitude faster than the other methods; once the credal-network build time is included, IBP, CDVE, and CCTE_{cm} remain substantially faster than exact LCN inference and are competitive with ARIEL.

Results on random LCNs: large DAGs. Figure 4 reports results on substantially larger dag LCNs, where Exact is no longer tractable and we therefore use ARIEL as the reference algorithm. Plain CCTE fails to scale beyond roughly 10 variables — the untruncated credal potentials grow too quickly — and is omitted from this plot. Among the remaining methods, CDVE again produces the tightest bounds, and its gap to ARIEL in fact *narrows* as the problem size grows, a favorable scaling signal for an inner-bound method. On running times the trend is reversed: IBP is consistently the fastest, whereas CDVE slows noticeably as the number of variables increases.

Results on realistic LCNs: Bayesian-network-derived instances. The top three blocks of Table 1 report results on the Alarm, Credit, and Engine LCNs from [?], using Exact as the reference. These are small in proposition count but hard to solve exactly (Exact requires between $\sim 1,000$ and $\sim 7,000$ seconds). CDVE consistently produces the tightest bounds among the credal-network methods and is in fact competitive with ARIEL on Engine ($\text{MAE}_l = 0.45$ vs. 0.39), while on Alarm and Credit ARIEL retains a clear edge because it operates directly on the original LCN rather than on its credal-network relaxation. The message-truncated CCTE_{cm} is a net win over plain CCTE on Alarm and Engine — tighter bounds *and* an order-of-magnitude speedup — so the ε -pruning heuristic pays off in practice.

Results on realistic LCNs: Supreme Court scaling instances. The bottom three blocks of Table 1 report results on the new Supreme Court chain-graph LCNs, using ARIEL as the reference. These instances are designed to test scalability: the dashes in the table indicate that the corresponding algorithm did not complete within the time and memory budget. Plain CCTE already times out

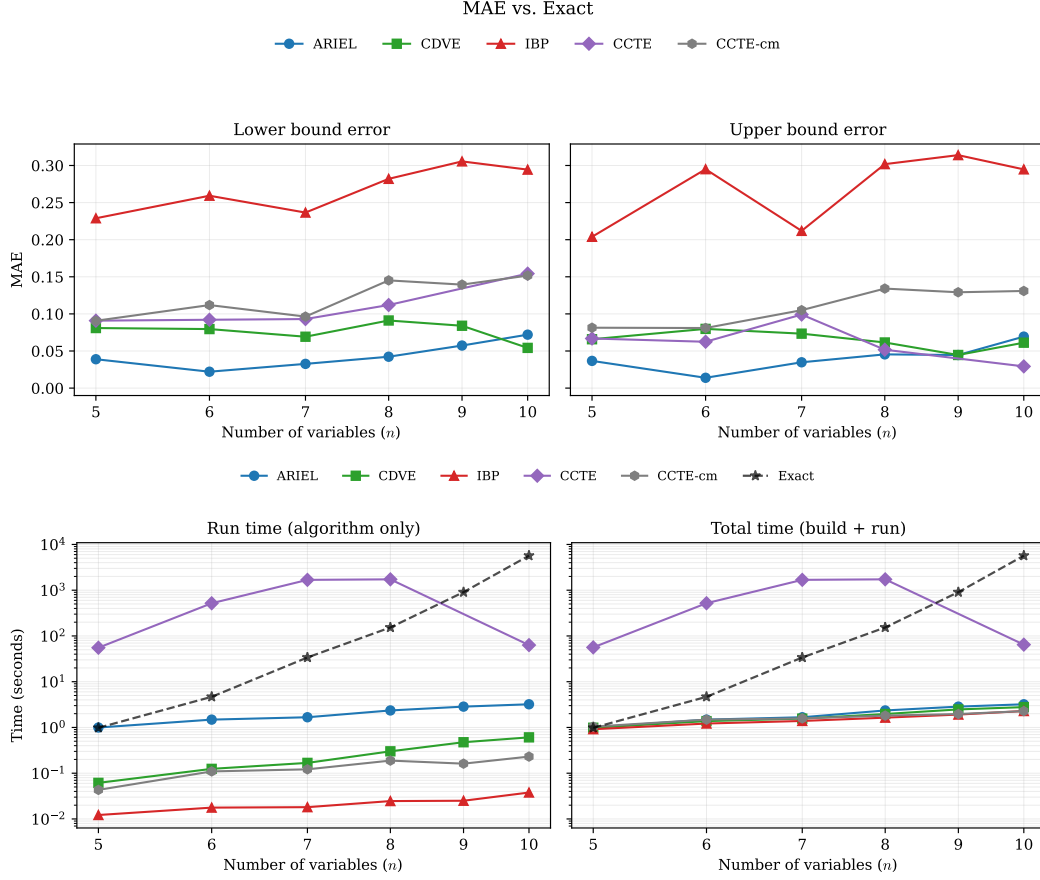


Figure 3: Results on small random dag LCNs. MAE vs Exact and running time (sec).

at $n=12$, CCTE_{cm} at $n=32$, and IBP at $n=40$; **CDVE is the only credal-network method that completes every instance**. On the largest benchmark (PScaling-n40), CDVE is roughly $2\times$ slower than ARIEL but provides inner — certified-contained — bounds, a qualitatively different guarantee that ARIEL does not offer.

Overall recommendation. Taken together, the experiments suggest a simple decision rule. CDVE is the method of choice when tight bounds matter and inner bounds are acceptable; IBP is the method of choice when speed dominates and looser outer bounds suffice; CCTE_{cm} offers a useful middle ground at moderate problem sizes. Plain CCTE is primarily valuable as a credal-network oracle on small instances.

5 Related Work

Credal networks [?] extend Bayesian networks by replacing precise conditional distributions with convex sets of distributions (credal sets). Exact inference in credal networks is NP-hard even for polytrees [?]. Approximate algorithms include the 2U and L2U schemes [?] for binary variables, the GL2U extension [?] for multi-valued variables, and the ApproxLP method [?]. Our work adapts and extends these algorithms to operate on the credal networks derived from chain graph LCN factorizations, with native support for multi-valued variables.

Probabilistic logic systems such as ProbLog [?] and DeepProbLog [?] combine logic programming with precise probabilities. Markov Logic Networks [?] attach weights to first-order logic formulas. Probabilistic Soft Logic [?] uses continuous truth values with hinge-loss potentials. LCNs differ from all of these by allowing *imprecise* probability specifications and operating in the propositional setting with few structural restrictions.

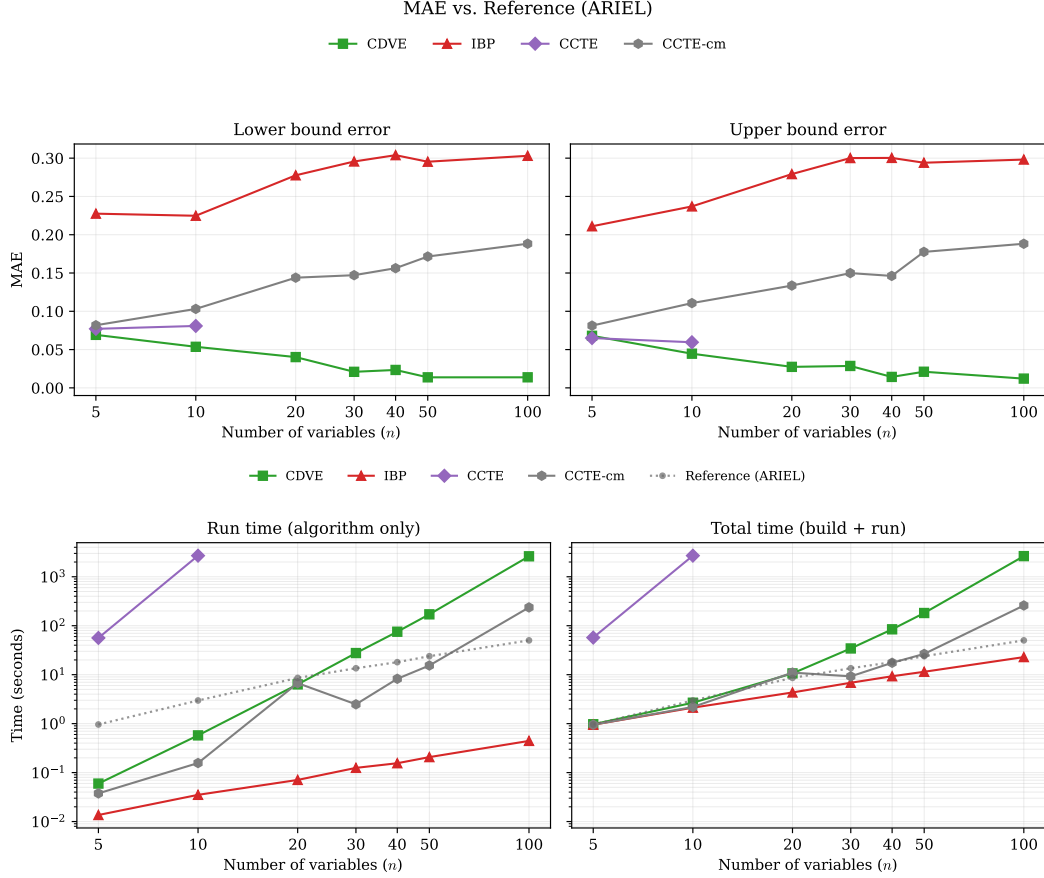


Figure 4: Results on large random dag LCNs. MAE vs ARIEL and running time (sec).

Previous work on LCN inference includes exact constrained non-linear optimization [?], the ARIEL message-passing scheme [?], and abductive (MAP) reasoning [?]. The chain graph factorization and Markov conditions were studied in [?]. Our work introduces three new algorithms that exploit this factorization for scalable approximate inference.

6 Conclusion

We revisited the problem of scalable approximate marginal inference in Logical Credal Networks by exploiting their chain graph factorization. Building on the observation that the local Markov condition decomposes global LCN inference into a collection of local credal network inference tasks, we bridged LCNs with the well-studied credal network formalism and brought a family of credal network techniques to bear on LCNs for the first time. On this basis, we developed three approximate inference algorithms that span complementary bound regimes: Credal Cluster Tree Elimination (CCTE) with ε -pruning and its message-truncated variant CCTE_{cm}, which compute outer bounds via bucket-tree elimination; Interval Belief Propagation (IBP), a fast outer-bound message-passing scheme with native support for multi-valued variables; and Coordinate Descent with Variable Elimination (CDVE), which produces inner bounds by optimizing over extreme points of the local credal sets. An extensive empirical evaluation on random LCNs, on benchmark LCNs derived from real Bayesian networks, and on a new family of causal chain graph LCNs from the Supreme Court analysis of [?], shows that CDVE yields the tightest bounds among credal-network methods — competitive with ARIEL on several instances and the only algorithm completing the largest Supreme Court benchmarks — while IBP is consistently the fastest at the cost of looser bounds and CCTE_{cm} offers a practical middle ground. The residual gap between our methods and ARIEL reflects the intrinsic cost of the chain-graph-to-credal-network relaxation ($\mathbb{K}(\mathbf{X}) \supseteq \mathcal{P}(\mathbf{X})$) and is a reasonable price for the

Instance	Algorithm	MAE _{lb}	MAE _{ub}	Build (sec)	Total (sec)	Ref (sec)
Alarm	CDVE	0.1708	0.0898	2.35	3.07	5094.99
	ARIEL	0.0700	0.0810	0.00	3.01	5094.99
	CCTE	0.1747	0.2222	2.16	967.10	5094.99
	CCTE _{cm}	0.1624	0.1143	2.14	8.88	5094.99
	IBP	0.1426	0.3158	2.26	2.34	5094.99
Credit	CDVE	0.2464	0.1459	2.48	3.20	6971.15
	ARIEL	0.0337	0.0490	0.00	2.75	6971.15
	CCTE	0.1328	0.1799	2.32	19.63	6971.15
	CCTE _{cm}	0.1215	0.0864	2.20	3.04	6971.15
	IBP	0.0919	0.0755	2.50	3.15	6971.15
Engine	CDVE	0.4513	0.0784	2.28	2.76	1059.54
	ARIEL	0.3922	0.0787	0.00	2.54	1059.54
	CCTE	0.4803	0.1482	2.13	3.04	1059.54
	CCTE _{cm}	0.3579	0.2183	2.04	8.79	1059.54
	IBP	0.4944	0.1211	2.29	2.33	1059.54
MScaling-n12	CDVE	0.1003	0.5013	2.81	3.90	5.60
	CCTE	-	-	-	-	-
	CCTE _{cm}	0.0738	0.2752	2.88	3.93	5.60
	IBP	0.3554	0.2748	2.86	2.92	5.60
BScaling-n32	CDVE	0.1148	0.2247	5.80	28.37	17.31
	CCTE	-	-	-	-	-
	CCTE _{cm}	-	-	-	-	-
	IBP	0.2317	0.2879	5.22	5.28	17.31
PScaling-n40	CDVE	0.1282	0.2631	6.97	62.63	31.41
	CCTE	-	-	-	-	-
	CCTE _{cm}	-	-	-	-	-
	IBP	-	-	-	-	-

Table 1: Results on the realistic LCNs. The top three blocks (Alarm, Credit, Engine) are the Bayesian-network-derived LCNs of [?], with Exact as the reference algorithm; the bottom three blocks (MScaling-n12, BScaling-n32, PScaling-n40) are the new Supreme Court chain-graph LCNs of [?], with ARIEL as the reference. Columns report MAE on the lower and upper posterior marginal bounds, build time, total time (build + inference), and reference-algorithm time (all in seconds). A dash indicates that the algorithm did not complete within the time or memory budget.

substantial scalability gains we observe. Together, these results significantly broaden the range of LCN instances that can be reasoned about in practice and open the door to imprecise-probability reasoning at scales previously out of reach.

Limitations. The factorization step requires the LCN structure to be a chain graph, which may not hold for all LCNs. The quality of IBP bounds on loopy graphs lacks formal guarantees. ApproxLP may converge to local optima. The scalability of all methods is ultimately limited by the treewidth of the underlying graph.

Future work. Promising directions include extending the algorithms to handle non-chain-graph structures, developing tighter convergence bounds for IBP on loopy graphs, combining multiple random restarts with ApproxLP to escape local optima, and applying these inference methods to downstream tasks such as neuro-symbolic AI and robust decision making under imprecise probabilities.

A Exact and Approximate Inference in LCNs

A.1 Exact Inference:

The exact inference problem in an LCN $\mathcal{L}$ with n propositions consists of computing lower and upper bounds on the posterior probability of a query formula ψ . This is formulated as a non-linear optimization problem over a vector of variables representing the probabilities of all 2^n interpretations of $\mathcal{L}$. The constraints ensure that these variables form a valid probability distribution, encode the probabilistic sentences in $\mathcal{L}$, and enforce conditional independence assumptions induced by the local Markov condition, while the objective function linearly represents $P(\psi)$ via indicator vectors and is

Algorithm 4 Credal Cluster Tree Elimination (CCTE)

Require: LCN $\mathcal{L}$, evidence $\mathbf{E} = \mathbf{e}$ ▷ DISTRIBUTE (DOWNWARD)
Ensure: Bounds $[P(X_i=x|\mathbf{e}), \bar{P}(X_i=x|\mathbf{e})]$
 for non-evidence variables X_i and states x
 1: Build credal network $\mathcal{C}$ using chain graph factorization of $\mathcal{L}$
 2: Build potentials from extreme points of each $\mathbb{K}(Y_j \mid \text{pa}(Y_j))$ and evidence indicators
 3: Build bucket tree $\mathcal{T}$: assign potentials, determine parent-child ▷ EXTRACT MARGINALS
▷ COLLECT (UPWARD)
 4: **for** each X_i in $\mathcal{T}$ (leaves to root) **do**
 5: $\psi_i \leftarrow \prod \{\text{local pots} \cup \{\lambda_{X_j} : X_j \in \text{ch}(X_i)\}\}$
 6: $\lambda_{X_i} \leftarrow \text{prune}(\sum_{X_i} \psi_i)$
 7: Send λ_{X_i} to parent
 8: **for** each X_i in $\mathcal{T}$ (root to leaves) **do**
 9: **for** each child X_j of X_i **do**
 10: $\psi \leftarrow \prod \{\pi_{X_i}, \text{local pots}, \{\lambda_{X_k} : X_k \in \text{ch}(X_i) \setminus \{X_j\}\}\}$
 11: $\pi_{X_j} \leftarrow \text{prune}(\sum_{X_i} \psi)$
 12: Send π_{X_j} to child X_j
 13: **for** each variable X_i **do**
 14: $\Phi_i \leftarrow \prod \{\text{local pots}, \{\lambda_{X_j} : X_j \in \text{ch}(X_i)\}, \pi_{X_i}\}$
 15: $\phi_i \leftarrow \sum_{\mathbf{Y} \setminus \{X_i\}} \Phi_i$
 16: $P(X_i=x) \leftarrow \min_{f \in \phi_i} f(x) / \sum_{x'} f(x')$
 17: $\bar{P}(X_i=x) \leftarrow \max_{f \in \phi_i} f(x) / \sum_{x'} f(x')$

402 minimized and maximized to obtain the desired bounds [?]. In practice, however, the algorithm does
 403 not scale beyond small problem instances [?].

404 A.2 Approximate Inference

405 More recently, an iterative message-passing scheme has been proposed for approximate inference in
 406 LCNs. The method, called ARIEL, operates on a factor-graph representation of an LCN, in which
 407 variable nodes correspond to LCN propositions and factor nodes correspond to LCN constraints
 408 defined over subsets of variables. Two types of messages are exchanged along the edges of the factor
 409 graph. Variable-to-factor messages convey the current lower and upper probability bounds of the
 410 variables, while factor-to-variable messages compute updated bounds by solving a local constrained
 411 non-linear optimization problem defined by the factor’s constraints. This computation additionally
 412 relies on an independence assumption among the neighboring variable nodes of the factor, which is
 413 encoded via quadratic equality constraints. Empirically, ARIEL has been shown to scale to larger
 414 problem instances while maintaining high-quality lower and upper bounds [?]. However, the
 415 approach is constrained by the size and complexity of the local non-linear programs required for
 416 factor-to-variable updates. Moreover, ARIEL remains the only available approximation scheme for
 417 marginal inference in LCNs.

418 B Detailed Description of the Algorithms

419 B.1 Credal Cluster Tree Elimination

420 We present next three inference algorithms that leverage the chain graph factorization of an LCN to
 421 compute marginal probability bounds. The first, Credal Cluster Tree Elimination (CCTE), extends
 422 classical cluster (bucket) tree elimination [? ? ?] to credal networks by propagating credal potentials,
 423 represented as finite sets of non-negative functions [?].

424 **Definition 7** (potential). A potential $\phi(\mathbf{Y})$ over variables $\mathbf{Y}$ is a finite set of non-negative real-valued
 425 functions $\{f_1, f_2, \dots, f_m\}$ on the joint domain of $\mathbf{Y}$. The product (or combination) of two potentials is
 426 defined by $\phi \cdot \psi = \{f \cdot g : f \in \phi, g \in \psi\}$, where $(f \cdot g)(\mathbf{y}, \mathbf{z}) = f(\mathbf{y}) \cdot g(\mathbf{z})$ over the joint domain $\mathbf{Y} \cup \mathbf{Z}$.
 427 The sum-marginal is defined by $\sum_X \phi(\mathbf{Y}) = \{\sum_x f(\mathbf{y}) : f \in \phi\}$, producing functions over $\mathbf{Y} \setminus \{X\}$.
 428 A pruning operation is defined by $\text{prune}(\phi) = \{f \in \phi : \nexists g \in \phi \setminus \{f\} \text{ s.t. } g(\mathbf{y}) \geq f(\mathbf{y}) \forall \mathbf{y}\}$.

429 Algorithm 4 describes our credal cluster tree elimination. Given an elimination ordering $\mathbf{o} =$
 430 $(X_1, X_2, \dots, X_n)$ and a set of initial potentials (one per credal factor plus evidence indicators): each
 431 potential is assigned to the bucket of the first variable in $\mathbf{o}$ that appears in the potential’s scope. For
 432 each bucket X_i , compute the *message scope*: the union of scopes of all potentials in the bucket
 433 (including received messages), minus X_i itself. The **parent** of X_i is the next variable in $\mathbf{o}$ that

434 appears in this message scope. The message scope is propagated to the parent’s effective scope. The
 435 result is a rooted tree where the last variable in the ordering is the root.

436 Process buckets in elimination order (leaves to root). At each bucket X_i :

- 437 1. **Gather**: local potentials + upward messages λ_{X_j} from all children X_j .
- 438 2. **Combine**: $\psi_i = \prod \{\text{gathered potentials}\}$.
- 439 3. **Marginalize**: $\lambda_{X_i} = \sum_{X_i} \psi_i$.
- 440 4. **Prune**: $\lambda_{X_i} \leftarrow \text{prune}(\lambda_{X_i})$.
- 441 5. **Send** λ_{X_i} to the parent bucket.

442 Process buckets in *reverse* elimination order (root to leaves). At each bucket X_i , for each child X_j :

- 443 1. **Gather**: the downward message π_{X_i} from X_i ’s own parent + local potentials in X_i ’s bucket
 444 + upward messages from all children of X_i *except* X_j .
- 445 2. **Combine**: $\psi = \prod \{\text{gathered potentials}\}$.
- 446 3. **Marginalize**: $\pi_{X_j} = \sum_{X_i} \psi$ (sum out X_i).
- 447 4. **Prune**: $\pi_{X_j} \leftarrow \text{prune}(\pi_{X_j})$.
- 448 5. **Send** π_{X_j} to child X_j .

449 After the distribute pass, every bucket has received information from the rest of the network via
 450 its parent’s downward message. We therefore extract all marginals: At each bucket X_i , the full
 451 information is:

$$\Phi_i = \underbrace{\text{local potentials}}_{\text{assigned in Step 1}} \cdot \underbrace{\prod_{j \in \text{children}(i)} \lambda_{X_j}}_{\text{from collect}} \cdot \underbrace{\pi_{X_i}}_{\text{from distribute}}.$$

452 Marginalize out all variables except X_i , then normalize each function and take component-wise
 453 min/max for the lower/upper bounds.

454 While exact CCTE preserves tight bounds, intermediate potentials can grow exponentially. Following
 455 [? ?], we approximate the potentials by fixing their cardinality to a relatively small number B .
 456 Specifically, we group the elements of $\phi(\mathbf{Y})$ into B clusters based on minimizing the Manhattan
 457 distance to each cluster’s centroid (i.e., minimize $\sum_{i=1}^m |p_i - r_i|$, where p_i and r_i are the i -th
 458 components of p and r , respectively). Then, for each cluster we replace its components with their
 459 mean values.

460 **Theorem 3** (complexity). *The time and space complexity of CCTE is $O(n \cdot C \cdot k^{w^*})$ where n is the*
 461 *number of variables, k bounds the domain size, w^* is the treewidth, and C is the maximum potential*
 462 *cardinality (which can be exponential). With potential approximation, C is bounded by B [? ?].*

463 C Additional Experiments

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: We discuss the details in Sections 3 and 4.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: We discuss the limitations in the experimental section.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: We discuss the details in Section 3.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We discuss the details in Section 4. Our benchmarks and code is included in the supplementary material.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: We plan to open-source the contribution as soon as we obtain legal approval. The code and benchmarks are included in the supplementary material.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The details are included in Section 4.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: We report average running times and mean absolute errors.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We discuss the details in Section 4.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- 671 • The paper should disclose whether the full research project required more compute
672 than the experiments reported in the paper (e.g., preliminary or failed experiments that
673 didn't make it into the paper).

674 **9. Code of ethics**

675 Question: Does the research conducted in the paper conform, in every respect, with the
676 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

677 Answer: [Yes]

678 Justification: We adhere to the NeurIPS code of ethics.

679 Guidelines:

- 680 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
681 Ethics.
682 • If the authors answer [No], they should explain the special circumstances that require a
683 deviation from the Code of Ethics.
684 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
685 eration due to laws or regulations in their jurisdiction).

686 **10. Broader impacts**

687 Question: Does the paper discuss both potential positive societal impacts and negative
688 societal impacts of the work performed?

689 Answer: [N/A]

690 Justification: This paper has no societal impact.

691 Guidelines:

- 692 • The answer [N/A] means that there is no societal impact of the work performed.
693 • If the authors answer [N/A] or [No], they should explain why their work has no societal
694 impact or why the paper does not address societal impact.
695 • Examples of negative societal impacts include potential malicious or unintended uses
696 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
697 (e.g., deployment of technologies that could make decisions that unfairly impact specific
698 groups), privacy considerations, and security considerations.
699 • The conference expects that many papers will be foundational research and not tied
700 to particular applications, let alone deployments. However, if there is a direct path to
701 any negative applications, the authors should point it out. For example, it is legitimate
702 to point out that an improvement in the quality of generative models could be used to
703 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
704 that a generic algorithm for optimizing neural networks could enable people to train
705 models that generate Deepfakes faster.
706 • The authors should consider possible harms that could arise when the technology is
707 being used as intended and functioning correctly, harms that could arise when the
708 technology is being used as intended but gives incorrect results, and harms following
709 from (intentional or unintentional) misuse of the technology.
710 • If there are negative societal impacts, the authors could also discuss possible mitigation
711 strategies (e.g., gated release of models, providing defenses in addition to attacks,
712 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
713 feedback over time, improving the efficiency and accessibility of ML).

714 **11. Safeguards**

715 Question: Does the paper describe safeguards that have been put in place for responsible
716 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
717 image generators, or scraped datasets)?

718 Answer: [N/A]

719 Justification: The paper does not pose any such risks.

720 Guidelines:

- 721 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [N/A]

Justification: The paper does not use existing assets.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not release new assets.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing nor research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: The core method development does not involve LLMs.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

Instance	Algorithm	MAE _{lb}	MAE _{ub}	Build	Run	Total	Exact
alarm.lcn	CDVE	0.1708	0.0898	2.35	0.72	3.07	5094.99
alarm.lcn	ARIEL	0.0700	0.0810	0.00	3.01	3.01	5094.99
alarm.lcn	CCTE	0.1747	0.2222	2.16	964.94	967.10	5094.99
alarm.lcn	CCTE _c <i>m</i>	0.1624	0.1143	2.14	6.75	8.88	5094.99
alarm.lcn	IBP	0.1426	0.3158	2.26	0.08	2.34	5094.99
asia.lcn	CDVE	0.0000	0.2228	1.69	0.26	1.95	87.89
asia.lcn	ARIEL	0.0131	0.2648	0.00	1.70	1.70	87.89
asia.lcn	CCTE _e	0.0555	0.2559	1.66	0.01	1.67	87.89
asia.lcn	CCTE _c <i>m</i>	0.0123	0.2326	1.65	3.58	5.22	87.89
asia.lcn	IBP	0.3131	0.2985	1.69	0.04	1.73	87.89
cancer.lcn	CDVE	0.0653	0.0272	1.04	0.07	1.11	0.42
cancer.lcn	ARIEL	0.0309	0.1301	0.00	1.44	1.44	0.42
cancer.lcn	CCTE _e	0.0653	0.1599	1.03	0.09	1.12	0.42
cancer.lcn	CCTE _c <i>m</i>	0.0545	0.0246	0.98	2.77	3.75	0.42
cancer.lcn	IBP	0.2853	0.2090	1.11	0.02	1.12	0.42
credit.lcn	CDVE	0.2464	0.1459	2.48	0.72	3.20	6971.15
credit.lcn	ARIEL	0.0337	0.0490	0.00	2.75	2.75	6971.15
credit.lcn	CCTE _e	0.1328	0.1799	2.32	17.31	19.63	6971.15
credit.lcn	CCTE _c <i>m</i>	0.1215	0.0864	2.20	0.83	3.04	6971.15
credit.lcn	IBP	0.0919	0.0755	2.50	0.65	3.15	6971.15
earthquake.lcn	CDVE	0.0041	0.0000	1.01	0.05	1.06	0.34
earthquake.lcn	ARIEL	0.0000	0.0006	0.00	0.72	0.72	0.34
earthquake.lcn	CCTE _e	0.0833	0.1051	0.94	0.01	0.94	0.34
earthquake.lcn	CCTE _c <i>m</i>	0.0028	0.0040	0.90	0.07	0.97	0.34
earthquake.lcn	IBP	0.3848	0.3817	1.02	0.02	1.04	0.34
engine.lcn	CDVE	0.4513	0.0784	2.28	0.48	2.76	1059.54
engine.lcn	ARIEL	0.3922	0.0787	0.00	0.00	2.54	1059.54
engine.lcn	CCTE _e	0.4803	0.1482	2.13	0.92	3.04	1059.54
engine.lcn	CCTE _c <i>m</i>	0.3579	0.2183	2.04	6.75	8.79	1059.54
engine.lcn	IBP	0.4944	0.1211	2.29	0.04	2.33	1059.54
hepatitis.lcn	CDVE	0.1955	0.2193	3.79	1.02	4.80	5601.91
hepatitis.lcn	ARIEL	0.1451	0.1256	0.00	3.03	3.03	5601.91
hepatitis.lcn	CCTE _c <i>m</i>	0.7647	0.8967	3.67	2.81	6.49	5601.91
hepatitis.lcn	IBP	0.2643	0.2917	3.97	1.20	5.17	5601.91
suicide.lcn	CDVE	0.2505	0.1036	2.05	0.53	2.58	5196.44
suicide.lcn	ARIEL	0.2500	0.0998	0.00	2.97	2.97	5196.44
suicide.lcn	CCTE _e	0.2585	0.2346	2.14	389.13	391.27	5196.44
suicide.lcn	CCTE _c <i>m</i>	0.2804	0.1512	2.00	0.33	2.32	5196.44
suicide.lcn	IBP	0.3549	0.3635	2.22	0.07	2.29	5196.44
tank.lcn	CDVE	0.3817	0.0568	2.55	0.96	3.50	5709.84
tank.lcn	ARIEL	0.3970	0.1140	0.00	3.91	3.91	5709.84
tank.lcn	CCTE _c <i>m</i>	0.9200	0.9260	2.41	0.46	2.87	5709.84
tank.lcn	CCTE _c <i>p</i>	0.9200	0.9260	2.47	0.38	2.85	5709.84
tank.lcn	IBP	0.4727	0.2917	2.69	0.09	2.78	5709.84
toy.lcn	CDVE	0.0891	0.0745	0.73	0.03	0.77	0.13
toy.lcn	ARIEL	0.0000	0.0000	0.00	0.50	0.50	0.13
toy.lcn	CCTE _e	0.0891	0.0515	0.73	0.11	0.84	0.13
toy.lcn	CCTE _{cm}	0.0667	0.0354	0.67	0.05	0.72	0.13
toy.lcn	IBP	0.1256	0.1530	0.80	0.01	0.82	0.13

Table 2: Results on the realistic LCNs. MAE vs Exact and running times.

Instance	Algorithm	MAE _{lb}		MAE _{ub}	Build	Total	Ref
benchmark_scaling_directions_n24_2.lcn	CDVE	0.1215	0.4370	14.27	28.42	42.69	12.71
benchmark_scaling_directions_n32_1.lcn	CDVE	0.1148	0.2247	5.80	22.57	28.37	17.31
benchmark_scaling_directions_n32_1.lcn	IBP	0.2317	0.2879	5.22	0.06	5.28	17.31
maintenance_scaling_n12_3.lcn	CDVE	0.1003	0.5013	2.81	1.09	3.90	5.60
maintenance_scaling_n12_3.lcn	CCTE_cm	0.0738	0.2752	2.88	1.05	3.93	5.60
maintenance_scaling_n12_3.lcn	IBP	0.3554	0.2748	2.86	0.06	2.92	5.60
maintenance_scaling_n6_1.lcn	CDVE	0.0963	0.3972	1.17	0.10	1.28	1.64
maintenance_scaling_n6_1.lcn	CCTE	0.0963	0.3391	1.28	697.66	698.94	1.64
maintenance_scaling_n6_1.lcn	CCTE_cm	0.1037	0.3194	1.29	0.13	1.43	1.64
maintenance_scaling_n6_1.lcn	IBP	0.2768	0.2742	1.18	0.02	1.20	1.64
maintenance_scaling_n9_2.lcn	CDVE	0.1049	0.4748	1.93	0.51	2.44	3.42
maintenance_scaling_n9_2.lcn	CCTE_cm	0.0810	0.2742	2.04	0.52	2.56	3.42
maintenance_scaling_n9_2.lcn	IBP	0.3291	0.2495	1.92	0.03	1.96	3.42
paper_scaling_directions_n35_2.lcn	CDVE	0.1442	0.4419	42.55	209.37	251.92	27.12
paper_scaling_directions_n40_1.lcn	CDVE	0.1282	0.2631	6.97	55.66	62.63	31.41

Table 3: Error metrics vs. ariel reference